PROJECT REPORT

Automated Voice-Based Attendance System

1. Project Overview

The Automated Voice-Based Attendance System is a Python-based utility designed to simplify the process of roll-calling. Instead of a teacher or administrator manually reading out names, the system uses Text-to-Speech (TTS) technology to announce names from a pre-defined list. The user then provides a simple keyboard input to mark the individual as present or absent.

2. Objectives

* To automate the verbal part of attendance marking.
* To create a lightweight, "non-fancy" tool that doesn't require complex databases.
* To provide a clear, formatted summary of attendance at the end of the session.

3. Requirements

The project is built using Python 3.x. To keep the "Build Your Own Project" (BYOP) spirit, dependencies are kept to a minimum:

* Library: pyttsx3 (Used for offline text-to-speech conversion).
* Hardware: Standard speakers/headphones and a keyboard.

4. Implementation Logic

The system follows a linear execution flow:

* Data Storage: A Python list stores the names of the students.
* Voice Engine: The pyttsx3 engine is initialized to convert string data into spoken audio.
* The Loop: The program iterates through the name list. For every name:
  * The system announces the name.
  * The program pauses using engine.runAndWait() to wait for user input.
  * The response is stored in a dictionary (mapping the name to the status).
* Reporting: Once the list is exhausted, the dictionary is parsed to display a final table.

5. Source Code

```
import pyttsx3

# Step 1: Initialize voice engine
engine = pyttsx3.init()
engine.setProperty('rate', 145) # Setting a natural speaking speed

# Step 2: Define the dataset (10 Indian Names)
students = [
    "Arjun", "Aditi", "Ishaan", "Sanya", "Vihaan",
    "Ananya", "Rohan", "Kavya", "Rahul", "Priya"
]

attendance_book = {}

print("--- Starting Voice Attendance System ---")

# Step 3: Main Logic Loop
for name in students:
    # Voice prompt
    print(f"\nChecking: {name}")
    engine.say(f"Is {name} present?")
    engine.runAndWait()

    # Input handling
    status_input = input("Enter 'y' for Yes / 'n' for No: ").lower().strip()

    if status_input == 'y':
        attendance_book[name] = "PRESENT"
```

```
    else:
        attendance_book[name] = "ABSENT"

# Step 4: Generate Summary
print("\n" + "="*30)
print("FINAL ATTENDANCE REPORT")
print("="*30)

for name, status in attendance_book.items():
    # Formatting to keep columns aligned
    print(f"{name:<12} | {status}")

print("="*30)
engine.say("Attendance session complete.")
engine.runAndWait()
```

6. Key Features

* Offline Functionality: Does not require an internet connection to speak.

* Case Insensitivity: The code handles "Y", "y", " yes ", or "Yes" correctly using string stripping and lowercasing.

* Clean UI: Uses basic string formatting (<12) to ensure the final report looks like a professional table in the console.

7. Future Scope

While this version is minimalist for the BYOP requirements, it can be scaled by:

* Importing names from an Excel or CSV file.

* Adding a timestamp to the final report.

* Integrating voice recognition so the user can say "Yes" instead of typing it.

```python
import pyttsx3

# Initialize the speaker engine
engine = pyttsx3.init()

# List of 10 Indian names
names = ["Arjun", "Aditi", "Ishaan", "Sanya", "Vihaan", "Ananya", "Rohan", "Kavya", "Rahul", "Priya"]

attendance_log = {}

print("--- Automated Voice Attendance ---")

for name in names:
    # System speaks the name
    print(f"\nSystem: Is {name} present?")
    engine.say(f"Is {name} present?")
    engine.runAndWait()

    # User inputs response
    response = input("Response (y/n): ").lower().strip()

    if response in ['y', 'yes']:
        attendance_log[name] = "Present"
```

```
PS C:\Users\singh\OneDrive\firstprogram.py>
```

```python
import pyttsx3

# Initialize the speaker engine
engine = pyttsx3.init()y

# List of 10 Indian names
names = ["Arjun", "Aditi", "Ishaan", "Sanya", "Vihaan", "Ananya", "Rohan", "Kavya", "Rahul", "Priya"]

attendance_log = {}

print("--- Automated Voice Attendance ---")

for name in names:
    # System speaks the name
    print(f"\nSystem: Is {name} present?")
    engine.say(f"Is {name} present?")
    engine.runAndWait()

    # User inputs response
    response = input("Response (y/n): ").lower().strip()

    if response in ['y', 'yes']:
        attendance_log[name] = "Present"
```

```
--- Automated Voice Attendance ---

System: Is Arjun present?
Response (y/n): y

System: Is Aditi present?
Response (y/n): y

System: Is Ishaan present?
Response (y/n): y

System: Is Sanya present?
Response (y/n):
```